Order Agent: Build Plan

Sales Profit Tracker. Prepared for the project owner.

Summary

A floating chat box that lets an Owner or Admin type or paste a Facebook order (text or a screenshot) and have an agent prepare the order. The agent finds or creates the customer, matches the products, and shows a draft. **Nothing is saved until the user presses Confirm.** The AI is Google Gemini, called from the backend only, with the key in `.env`.

Decisions already made

- Save only after the user confirms the draft.
- The chat box floats on every page.
- Orders come from Facebook, so input is mostly pasted Messenger text and screenshots, often Bengali or mixed Bengali/English.
- AI provider: Gemini, with the key already stored as GEMINI_API_KEY in .env (git-ignored).

Rules that apply to every part

- **Work one part at a time.** After a part is finished I stop and wait for you to say start the next one.
- The Gemini key stays on the backend. The browser never sees it.
- The agent can only do what the logged-in user is allowed to do (roles are respected).
- The model only extracts details. Matching customers and products, and all saving, is done by our own backend code.
- Each part is tested, and committed on its own, with no AI attribution lines in the commit message.

Overview of the parts

Part	What it delivers	Visible to you?
1	Backend connection to Gemini (text in, structured details out)	Test endpoint only
2	Matching: find/create customer, match products, build a draft order	Test endpoint only
3	Floating chat box with a draft card and Confirm / Cancel	Yes
4	Screenshot input (paste or attach an image)	Yes
5	Follow-up edits in chat ("make it 3", "change the address")	Yes
6	Safety, activity log, limits, polish	Yes

The parts in detail

Part 1: Gemini connection (backend)

Goal: The backend can send text to Gemini and get back clean structured data.

What gets built

- Pass GEMINI_API_KEY from .env into the backend container (docker-compose).
- A small AI client class, behind an interface so another provider can replace Gemini later.
- A prompt that asks for JSON only: customer name, phone, address, items (name, quantity, price), platform, delivery charge, notes.
- Understands Bengali digits and mixed Bengali/English text.
- Clear errors when the key is missing, the quota is hit, or Gemini is down.
- A temporary test endpoint, for Owner/Admin only, to try it with sample text.

Finished when: pasting a sample Facebook order into the test endpoint returns correct structured data.

Part 2: Matching and draft order (backend)

Goal: Turn the extracted data into a ready-to-confirm draft that uses your real customers and products.

What gets built

- Customer: look up by phone number; if none exists, mark as a new customer to be created.
- Products: match the names to your product list (fuzzy match), fill in price and cost, and flag anything unclear.
- A draft with a list of problems: missing phone, unknown product, ambiguous match, not enough stock.
- Two endpoints: *prepare draft* (saves nothing) and *confirm draft* (creates the customer if needed and the order, using the existing order logic so order numbers, stock and activity work as normal).
- Cost on a line is only set by Owner/Admin, same as the order form today.

Finished when: text in produces a draft; confirming it creates the customer and the order exactly like the normal form would.

Part 3: Floating chat box (frontend)

Goal: You can use the agent from any page.

What gets built

- A floating button at the corner of every page that opens a chat panel.
- Type or paste an order; the agent replies with a draft card showing customer, items, totals and any warnings.
- Confirm and Cancel buttons; Confirm saves, and the card links to the new order number.
- Loading and error states; the panel is hidden for roles that may not create orders.
- Works on mobile width.

Finished when: you can paste a Facebook order in the chat, see the draft, press Confirm and find the order in Orders.

Part 4: Screenshot input

Goal: Paste or attach a Messenger screenshot instead of typing.

What gets built

- Attach button and Ctrl+V paste of an image in the chat box.
- Image sent to Gemini's vision model; same extraction and matching as text.
- Size and type limits, with the image resized before sending to keep it fast and cheap.
- Images are not stored; they are used for the request only.

Finished when: a screenshot of a Facebook conversation produces a correct draft.

Part 5: Follow-up edits in chat

Goal: Fix a draft by talking to it instead of retyping.

What gets built

- Messages like "make it 3", "use Dhaka address", "add delivery charge 60" update the draft.
- The draft card refreshes after each change and still needs Confirm.
- Missing details are asked back as questions ("What is the phone number?").

Finished when: a draft can be corrected in two or three short messages and then confirmed.

Part 6: Safety and polish

Goal: Make it safe and reliable for daily use.

What gets built

- Orders created by the agent are marked in the Activity log.
- Rate limit per user to avoid accidental heavy use.
- Duplicate guard: warn if the same customer and items were already ordered a moment ago.
- Backend tests for extraction parsing, matching and role checks.
- Short note in the README/RUNBOOK on setting up the key; remove the temporary test endpoint.
- Reminder to replace the API key that was pasted in chat.

Finished when: all tests pass and the feature is ready for daily use.

Costs and privacy

- Gemini API has a free tier with rate limits, which should cover a small shop. Check Google's current limits and pricing.
- On the free tier, Google may use prompts to improve its products. Customer names, phones and addresses would be included. Enable billing on the key if that is a concern.
- The Gemini Plus app subscription does not include API access; the key from Google AI Studio is separate.

Risks to keep in mind

- The model can misread a digit or a product name. This is why every draft must be confirmed by a person.
- Bengali handwriting-style or very cropped screenshots may extract poorly; the draft will show what is missing instead of guessing.
- A leaked key can be used by others; it is git-ignored and server-side only.